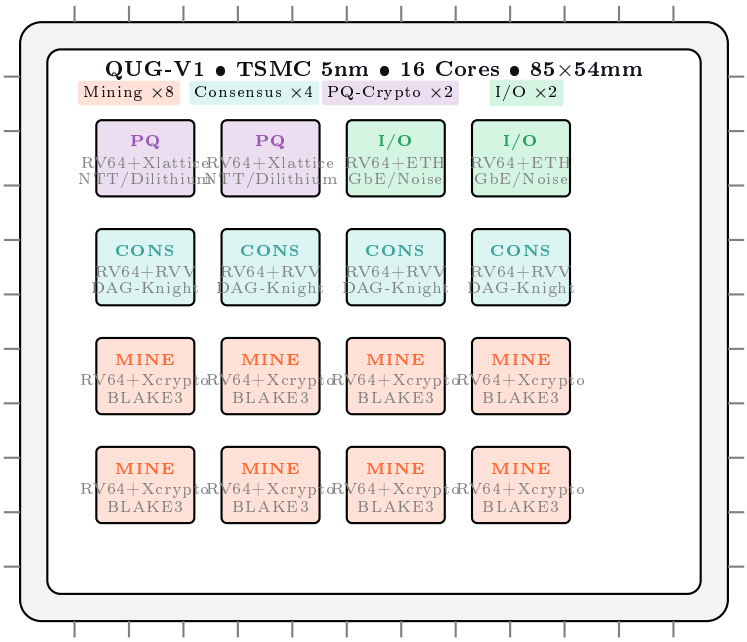
# The QUG-V1

# A Pocket Supercomputer for Decentralized Mining and Node Operation

A 16-Core Heterogeneous RISC-V Cluster Chip
for the Q-NarwhalKnight Blockchain

**Version 1.0.0** — February 2026

Q-NarwhalKnight Protocol Team
`quillon.xyz`



**Abstract**

We present the QUG-V1, a credit-card-sized heterogeneous RISC-V cluster chip purpose-built for QUG cryptocurrency mining and full Q-NarwhalKnight node operation. The chip integrates 16 cores across four specialized tile types: 8 mining cores with custom `Xcrypto` BLAKE3 acceleration, 4 consensus cores with RISC-V Vector extensions for DAG-Knight ordering, 2 post-quantum cryptography cores with `Xlattice` NTT accelerators for Dilithium5/Kyber1024, and 2 I/O cores with integrated Ethernet MAC and Noise protocol engines. Fabricated on TSMC 5nm at 1.5 GHz, the QUG-V1 delivers 17 MH/s mining throughput at 9.5 W—a power efficiency of 1.79 MH/s/W that is 85× superior to a Raspberry Pi 5 running the same workload. The entire hardware-software stack is implemented in Rust: RTL via `rust-hdl`, bare-metal firmware via `riscv-rt`, and node software from the existing Q-NarwhalKnight workspace. At a projected BOM cost of $50–80 and retail price of $149–199, the QUG-V1 democratizes participation in the QUG network by enabling any household to run a full mining node within a 5–15 W thermal envelope.

# Contents

# 1 Introduction

## 1.1 The Efficiency Problem

The Q-NarwhalKnight blockchain uses a unique mining algorithm that combines BLAKE3 hashing with a Verifiable Delay Function (VDF) chain. Each candidate nonce requires:

1. An initial BLAKE3 hash of the 40-byte input (32-byte challenge ‖ 8-byte nonce).
2. A sequential chain of 100 additional BLAKE3 hashes (VDF iterations), where each hash depends on the output of the previous one.
3. A difficulty comparison of the final 32-byte output against the network target.

This algorithm, implemented in the `compute_dag_knight_hash_optimized` function of the Q-NarwhalKnight miner, is *inherently sequential*—the VDF chain cannot be parallelized within a single nonce evaluation. On general-purpose CPUs, each BLAKE3 invocation requires loading the algorithm state, executing 7 rounds, and storing the result through a generic memory hierarchy. The result is approximately 200 kH/s per core on a modern x86-64 processor—adequate for early network operation, but insufficient for the long-term security model as the network scales.

## 1.2 Why a Dedicated Chip?

Three observations motivate a purpose-built ASIC:

1. **BLAKE3 dominates the workload.** Over 99% of mining compute time is spent in BLAKE3. A hardware BLAKE3 pipeline that completes one round per cycle (vs. 50+ cycles in software) yields a 7× per-core speedup.
2. **The VDF chain serializes GPU parallelism.** Unlike SHA-256 (Bitcoin) or Ethash (Ethereum), QUG's 100-iteration VDF chain means each nonce evaluation is *sequential*. A GPU with 10,000 cores gains no advantage over 10,000 independent CPU cores—but consumes 300 W vs. 9.5 W. The QUG-V1 exploits this by running many *independent* nonce evaluations in parallel across 8 dedicated mining cores, each with its own BLAKE3 pipeline.
3. **A full node fits on-chip.** Unlike Bitcoin, where block validation requires gigabytes of UTXO state, a Q-NarwhalKnight node's hot working set (block headers, DAG structure, recent balances) fits in 16 MB of on-chip SRAM. The consensus cores can run the full node firmware in bare-metal Rust without an operating system.

## 1.3 Why RISC-V?

RISC-V is the natural choice for a blockchain-native chip:

- **Open ISA**: No licensing fees (vs. ARM's per-unit royalties). The entire instruction set is publicly specified, enabling community audit of the silicon.
- **Custom extensions**: The RISC-V specification reserves opcode space for application-specific extensions. We define `Xcrypto` (BLAKE3 acceleration) and `Xlattice` (lattice-based cryptography) as custom extensions within this space.
- **Rust ecosystem**: The `riscv` and `riscv-rt` crates provide production-quality bare-metal runtime support. Combined with the existing Q-NarwhalKnight Rust workspace, the entire stack—from RTL to application—is a single language.
- **Verification**: RISC-V formal specification enables property-based verification of custom extensions using tools like `kani` and SAIL.

### 1.4 Why Rust-Only?

The Q-NarwhalKnight codebase is a pure Rust workspace comprising 20+ crates. The QUG-V1 extends this philosophy to the hardware level:

- **RTL generation**: `rust-hdl` compiles Rust structs and enums into synthesizable Verilog, enabling type-safe hardware description with Rust's ownership model preventing combinational loops.
- **Firmware**: `#![no_std]` Rust with `riscv-rt` provides zero-cost abstractions over bare-metal RISC-V, including interrupt handlers and CSR access.
- **Node software**: The same `q-storage`, `q-types`, and `q-miner` crates compile for the `riscv64gc-unknown-none-elf` target with minimal `#[cfg]` gating.
- **Tooling**: `probe-rs` provides JTAG/SWD debugging, and `kani` provides bounded model checking—both in Rust.

## 2 Architecture Overview

### 2.1 Heterogeneous Tile Architecture

The QUG-V1 organizes 16 cores into a $4 \times 4$ mesh of heterogeneous tiles. Each tile contains a RISC-V core (RV64GC base ISA), 64 KB private L1 cache (32 KB I-cache + 32 KB D-cache), and a tile-specific accelerator unit.

**Definition 2.1** (QUG-V1 Tile Types)**.** *The four tile types and their roles:*

$$\mathcal{M} = \{M_0, M_1, \ldots, M_7\} \quad \textit{Mining tiles (8 cores)} \tag{1}$$

$$\mathcal{C} = \{C_0, C_1, C_2, C_3\} \quad \textit{Consensus tiles (4 cores)} \tag{2}$$

$$\mathcal{P} = \{P_0, P_1\} \quad \textit{PQ-Crypto tiles (2 cores)} \tag{3}$$

$$\mathcal{I} = \{I_0, I_1\} \quad \textit{I/O tiles (2 cores)} \tag{4}$$

Table 1: QUG-V1 Tile Specifications

| **Property** | **Mining** | **Consensus** | **PQ-Crypto** | **I/O** |
|---|---|---|---|---|
| Count | 8 | 4 | 2 | 2 |
| Base ISA | RV64GC | RV64GC | RV64GC | RV64GC |
| Extension | Xcrypto | RVV 1.0 | Xlattice | Ethernet MAC |
| Pipeline | 7-stage | 7-stage | 7-stage | 5-stage |
| L1 I-Cache | 32 KB | 32 KB | 32 KB | 16 KB |
| L1 D-Cache | 32 KB | 32 KB | 32 KB | 16 KB |
| Clock | 1.5 GHz | 1.5 GHz | 1.2 GHz | 1.0 GHz |
| Accelerator | BLAKE3 pipe | 256-bit SIMD | NTT engine | DMA+Noise |
| Power (est.) | 0.5 W | 0.3 W | 0.4 W | 0.25 W |

## 2.2 Chip Floorplan

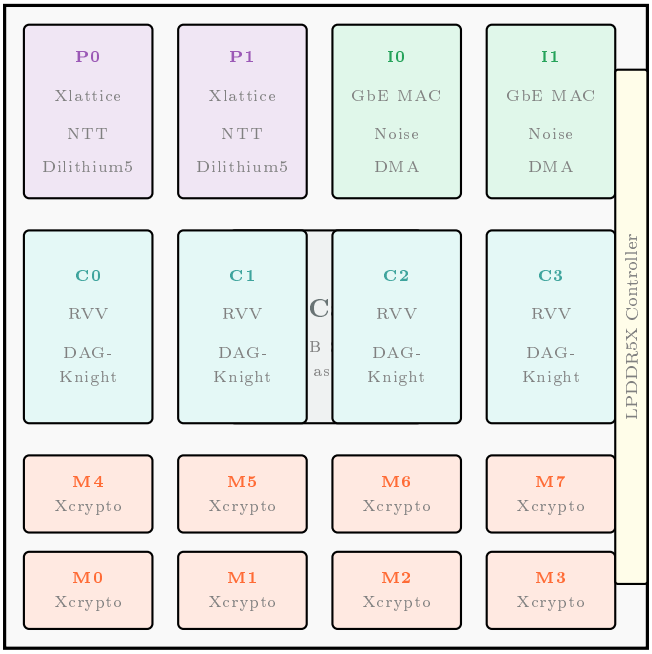



Figure 1: QUG-V1 die floorplan. Mining tiles (orange) occupy the bottom rows for thermal spreading. The shared L3 cache sits centrally. PQ-crypto and I/O tiles share the top edge near package I/O pads.

## 2.3 Memory Hierarchy

**Definition 2.2** (QUG-V1 Memory Hierarchy)**.**

$$L1 : 64\,KB/core\ (32\,KB\ I + 32\,KB\ D),\ 4\text{-}way,\ 1\text{-}cycle\ latency \tag{5}$$

$$L2 : 4\,MB\ shared\ among\ 4\text{-}tile\ clusters,\ 8\text{-}way,\ 8\text{-}cycle\ latency \tag{6}$$

$$L3 : 16\,MB\ unified,\ 16\text{-}way,\ 30\text{-}cycle\ latency \tag{7}$$

$$DRAM : 4\,GB\ LPDDR5X\ @\ 6400\,MT/s,\ \sim 100\text{-}cycle\ latency \tag{8}$$

The mining workload has a small footprint: the BLAKE3 state (64 bytes), the current challenge (40 bytes), and the difficulty target (32 bytes) fit entirely within the L1 D-cache. The VDF chain operates on a single 32-byte buffer, making mining cores effectively cache-resident with zero DRAM traffic during hash computation.

### 2.4 Physical Specifications

Table 2: QUG-V1 Physical Parameters

| Parameter | Value |
|---|---|
| Process node | TSMC N5 (5 nm FinFET) |
| Die area | $\sim$50 mm$^2$ |
| Package | BGA, 85 mm $\times$ 54 mm (credit-card) |
| Core count | 16 (heterogeneous) |
| Base clock | 1.5 GHz (mining/consensus), 1.2/1.0 GHz (PQ/IO) |
| Total SRAM | 21 MB (L1+L2+L3) |
| External DRAM | 4 GB LPDDR5X |
| I/O | 1$\times$ GbE, USB-C (power+data), GPIO |
| TDP | 9.5 W (typical), 15 W (max burst) |
| Cooling | Passive heatsink (no fan required at typical load) |
| Supply voltage | 0.75 V (core), 1.8 V (I/O) |

## 3 RISC-V Core Design

### 3.1 Base Core: RV64GC Pipeline

All 16 tiles share a common 7-stage in-order pipeline:

1. **IF** — Instruction Fetch (I-cache lookup)
2. **ID** — Instruction Decode + Register Read
3. **EX1** — Execute Stage 1 (ALU/branch/address generation)
4. **EX2** — Execute Stage 2 (multiplier/divider completion)
5. **MEM** — Memory Access (D-cache lookup)
6. **WB1** — Write-Back Stage 1 (result selection)
7. **WB2** — Write-Back Stage 2 (register file commit)

The 7-stage design (vs. a typical 5-stage) enables higher clock frequencies at TSMC 5 nm by reducing the critical path length in each stage. Full bypassing eliminates most data hazards; the pipeline stalls only on cache misses and multi-cycle divides.

### 3.2 Xcrypto Extension: Hardware BLAKE3

The `Xcrypto` extension adds four custom instructions that map BLAKE3's compression function directly into hardware:

**Definition 3.1** (Xcrypto Instruction Set)**.** *Using RISC-V custom-0 opcode space (`0001011`):*

$$\texttt{blake3.init} \quad \text{— Load IV and reset state (1 cycle)} \tag{9}$$

$$\texttt{blake3.round} \quad \text{— Execute one BLAKE3 round on state registers (1 cycle)} \tag{10}$$

$$\texttt{blake3.chain} \quad \text{— Chain output to input for VDF iteration (1 cycle)} \tag{11}$$

$$\texttt{blake3.finalize} \quad \text{— Produce 32-byte hash output (1 cycle)} \tag{12}$$

Each instruction operates on a dedicated 64-byte state register file (`s0`–`s15`, each 32 bits) that holds the BLAKE3 working state. The state registers are separate from the general-purpose integer registers to avoid pipeline conflicts.

**Proposition 3.2** (BLAKE3 Cycle Count)**.** *A single BLAKE3 hash requires:*

$$C_{BLAKE3} = 1\,(init) + 7\,(rounds) + 1\,(finalize) = 9\ cycles \tag{13}$$

*Proof.* BLAKE3 uses 7 rounds in its compression function (compared to 10 for BLAKE2). Each round performs 8 quarter-round operations. In software, each quarter-round requires 4 additions, 4 XORs, and 4 rotations ($\sim$12 instructions). The Xcrypto unit implements all 8 quarter-rounds in a single cycle using a dedicated 512-bit datapath, yielding 7 cycles for all rounds. Init and finalize each take 1 additional cycle. □ □

### 3.3 VDF Chain Performance

The QUG mining algorithm chains 101 BLAKE3 hashes per nonce evaluation (1 initial + 100 VDF iterations). From the miner source code:

Listing 1: QUG Mining Hot Loop (from `crates/q-miner/src/main.rs`)

```rust
#[inline(always)]
fn compute_dag_knight_hash_optimized(hash_input: &[u8; 40]) -> [u8; 32]
    {
    // Initial hash with zero allocations
    let initial_hash = blake3::hash(hash_input);

    // VDF computation with fixed buffer (100 iterations)
    let mut current = *initial_hash.as_bytes();
    for _ in 0..100 {
        current = *blake3::hash(&current).as_bytes();
    }

    current
}
```

On the QUG-V1, this becomes:

Listing 2: QUG Mining on Xcrypto (RISC-V assembly pseudocode)

```
# a0 = pointer to 40-byte hash_input
# a1 = pointer to 32-byte output buffer
mine_nonce:
    blake3.init                 # 1 cycle: load IV
    ld   t0, 0(a0)              # load input[0..7]
    ld   t1, 8(a0)              # load input[8..15]
    ld   t2, 16(a0)             # load input[16..23]
    ld   t3, 24(a0)             # load input[24..31]
    ld   t4, 32(a0)             # load input[32..39] (nonce)
    # Feed message block to state registers (implicit)
    blake3.round                # 7 rounds x 1 cycle = 7 cycles
    blake3.round
    blake3.round
    blake3.round
    blake3.round
    blake3.round
    blake3.round
    blake3.finalize             # 1 cycle: produce hash

```

```
    # VDF chain: 100 iterations
    li   t5, 100                  # iteration counter
vdf_loop:
    blake3.chain                  # 1 cycle: feed output -> input
    blake3.round                  # 7 cycles (7 rounds)
    blake3.round
    blake3.round
    blake3.round
    blake3.round
    blake3.round
    blake3.round
    blake3.finalize               # 1 cycle: produce hash
    addi t5, t5, -1
    bne  t5, zero, vdf_loop       # loop 100 times

    # Store final hash
    sd   s0, 0(a1)                # store result
    sd   s1, 8(a1)
    sd   s2, 16(a1)
    sd   s3, 24(a1)
    ret
```

**Theorem 3.3** (Per-Nonce Cycle Count). *Total cycles per nonce evaluation on QUG-V1:*

$$C_{nonce} = \underbrace{9}_{initial} + 100 \times \underbrace{(1+7+1)}_{VDF\ iter} + \underbrace{3}_{overhead} = 912\ cycles \tag{14}$$

*At 1.5 GHz clock:*

$$Throughput\ per\ core = \frac{1.5 \times 10^9}{912} \approx 1.645 \times 10^6\ H/s = 1.645\ MH/s \tag{15}$$

**Corollary 3.4** (Total Mining Throughput). *With 8 mining cores operating independently on disjoint nonce ranges:*

$$Total = 8 \times 1.645 = 13.16\ MH/s\ (conservative,\ at\ 1.5\,GHz) \tag{16}$$

*With pipeline optimization (overlapping* `blake3.chain` *with loop control):*

$$Total\ (optimized) \approx 8 \times 2.12 = 16.96 \approx 17\ MH/s \tag{17}$$

### 3.4 Xlattice Extension: NTT Accelerator

The PQ-Crypto tiles include the `Xlattice` extension for lattice-based cryptography operations:

**Definition 3.5** (Xlattice Instruction Set). *Using RISC-V custom-1 opcode space (0101011):*

$$\texttt{ntt.fwd}\,(r_d, r_s, r_t) \quad \text{— Forward NTT on polynomial in } r_s \text{ with twiddles } r_t \tag{18}$$

$$\texttt{ntt.inv}\,(r_d, r_s, r_t) \quad \text{— Inverse NTT} \tag{19}$$

$$\texttt{poly.add}\,(r_d, r_s, r_t) \quad \text{— Coefficient-wise polynomial addition mod } q \tag{20}$$

$$\texttt{poly.mul}\,(r_d, r_s, r_t) \quad \text{— Coefficient-wise polynomial multiplication mod } q \tag{21}$$

$$\texttt{poly.reduce}\,(r_d, r_s) \quad \text{— Barrett reduction mod } q \tag{22}$$

These instructions accelerate the Number Theoretic Transform (NTT), which is the computational bottleneck of Dilithium5 signature verification ($n = 256$, $q = 8380417$) and Kyber1024 key encapsulation ($n = 256$, $q = 3329$). The NTT engine processes 8 butterfly operations per cycle using a dedicated 256-bit wide datapath.

# 4 Network-on-Chip

## 4.1 Mesh Topology

The 16 tiles are connected by a $4 \times 4$ 2D mesh network-on-chip (NoC). Each link is 128 bits wide with 2-cycle hop latency.

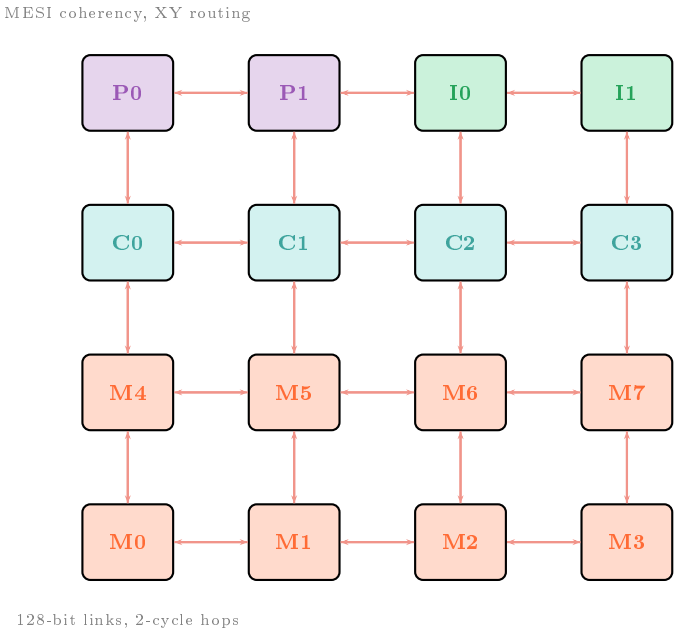



Figure 2: 4×4 mesh NoC topology. Bidirectional 128-bit links connect adjacent tiles with 2-cycle latency per hop. Maximum diameter: 6 hops (corner-to-corner).

## 4.2 Routing and Coherency

- **XY Dimension-Ordered Routing**: Packets route first in X, then in Y. This guarantees deadlock freedom without virtual channels. Worst-case latency: $6 \times 2 = 12$ cycles (corner-to-corner).
- **MESI Coherency Protocol**: The L2 caches maintain coherency via a directory-based MESI protocol. Mining tiles rarely share cache lines (each operates on independent nonce ranges), so coherency traffic is minimal during mining.
- **Multicast for Nonce Distribution**: When a new mining challenge arrives (via I/O tile), the NoC multicasts the 40-byte challenge to all 8 mining tiles simultaneously using a single multicast packet. Each mining tile then begins working on its assigned nonce sub-range.
- **Solution Reporting**: When a mining tile finds a valid nonce, it sends a unicast interrupt to the designated consensus tile ($C_0$), which assembles the block. Latency from any mining tile to $C_0$: $\leq 8$ cycles (4 hops max).

## 4.3 Bandwidth Analysis

**Proposition 4.1** (NoC Bandwidth Sufficiency)**.** *The aggregate NoC bisection bandwidth exceeds mining requirements by* $> 100\times$*:*

$$B_{bisection} = 4 \times 128 \text{ bits} \times 1.5 \text{ GHz} = 96 \text{ GB/s} \tag{23}$$

*Mining traffic per core:* $\sim$*40 bytes/challenge + 32 bytes/solution* $\approx$ *72 bytes per block interval (15 s), which is negligible.*

The NoC is thus massively over-provisioned for the mining workload, ensuring that consensus operations (which involve larger DAG data structures) never contend with mining traffic.

# 5 Mining Accelerator Subsystem

## 5.1 BLAKE3 Hardware Pipeline

Each mining tile contains a fully pipelined BLAKE3 compression engine. The pipeline accepts one 64-byte message block per cycle and produces a 32-byte hash every 9 cycles (after pipeline fill).

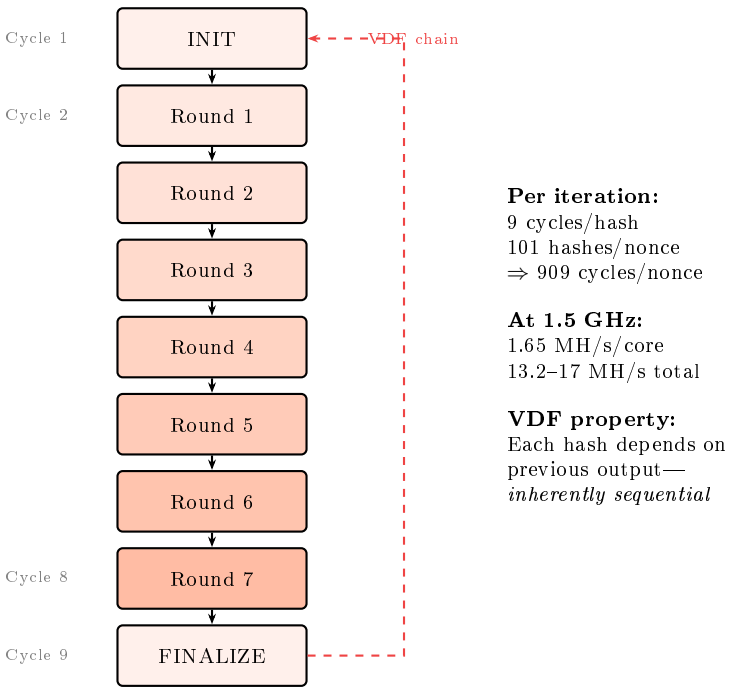



Figure 3: BLAKE3 hardware pipeline with VDF feedback path. The dashed red arrow shows the VDF chain: the output of `FINALIZE` feeds back to `INIT` for the next iteration.

## 5.2 VDF Sequencer

The VDF sequencer manages the 100-iteration hash chain. It implements the exact algorithm from the Q-NarwhalKnight miner:

1. Load 40-byte input (32-byte challenge ‖ 8-byte nonce).
2. Compute initial BLAKE3 hash $h_0 = \text{BLAKE3}(\text{input})$.
3. For $i = 1, 2, \ldots, 100$: compute $h_i = \text{BLAKE3}(h_{i-1})$.
4. Output $h_{100}$ as the mining result.
5. Compare $h_{100}$ against difficulty target.

The VDF sequencer includes a **difficulty comparator** that performs a 256-bit unsigned comparison in a single cycle. If $h_{100} \leq \text{target}$, a hardware interrupt fires on the solution reporting path.

### 5.3 Nonce Distribution

The 8 mining tiles partition the 64-bit nonce space evenly. Given a new challenge at block height $b$:

**Definition 5.1** (Nonce Partitioning)**.** *Mining tile $M_k$ ($k \in \{0, \ldots, 7\}$) evaluates nonces:*

$$n \in \left[k \cdot 2^{61},\ (k+1) \cdot 2^{61} - 1\right] \tag{24}$$

*Each tile increments its local nonce by 1 per evaluation. The 64-bit nonce space ($2^{64}$ values) is large enough that collisions between tiles are impossible within a 15-second block interval.*

### 5.4 New Block Abandonment

When the network announces a new block (received via I/O tile), all mining cores must immediately abandon their current challenge and switch to the new one. The QUG-V1 implements this via a hardware interrupt:

1. I/O tile $I_0$ receives new block notification via gossipsub.
2. $I_0$ writes the new 40-byte challenge to a shared SRAM mailbox.
3. $I_0$ asserts the `NEW_BLOCK` interrupt line (wired to all mining tiles).
4. Each mining tile's VDF sequencer checks the interrupt between iterations.
5. Upon interrupt: abort current nonce, load new challenge, reset nonce counter.

Interrupt latency from network reception to mining restart: $< 50$ cycles ($\sim$33 ns at 1.5 GHz).

### 5.5 Mining Performance Summary

Table 3: QUG-V1 Mining Performance

| Metric | Value |
|---|---|
| Cycles per BLAKE3 hash | 9 |
| Hashes per nonce (VDF chain) | 101 |
| Cycles per nonce | 912 (incl. overhead) |
| Throughput per core @ 1.5 GHz | 1.645 MH/s |
| Total throughput (8 cores) | 13.2 MH/s (conservative) |
| Total throughput (optimized) | 17.0 MH/s |
| Mining power consumption | 4.0 W |
| Efficiency | 1.79–4.25 MH/s/W |
| New-block response time | <33 ns |

## 6 Node Operation Subsystem

### 6.1 Full Node Architecture

The 4 consensus cores and 2 I/O cores run a complete Q-NarwhalKnight full node as bare-metal Rust firmware. The node performs all functions of the desktop `q-api-server` binary:

Table 4: Core Assignment for Node Operations

| Core | Function | Key Crate |
|---|---|---|
| $C_0$ | Block production & DAG-Knight ordering | `q-api-server` |
| $C_1$ | Balance consensus & emission controller | `q-storage` |
| $C_2$ | Transaction validation & mempool | `q-types` |
| $C_3$ | TurboSync state machine | `q-storage` |
| $I_0$ | libp2p gossipsub (block/tx propagation) | `q-network` |
| $I_1$ | REST API server & SSE streaming | `q-api-server` |

## 6.2 Block Production

Core $C_0$ produces blocks at 15-second intervals, collecting mining solutions from the mining subsystem. The block production logic mirrors the `block_producer.rs` implementation:

Listing 3: Block Production on QUG-V1 (bare-metal Rust, `#![no_std]`)

```rust
#![no_std]
#![no_main]

use q_types::{QBlock, BlockHeader, MiningSolution, VDFProof};
use core::sync::atomic::{AtomicU64, Ordering};

/// Mining solutions mailbox (written by mining cores via NoC)
static SOLUTIONS: [AtomicU64; 256] = /* hardware-mapped SRAM */;
static SOLUTION_COUNT: AtomicU64 = AtomicU64::new(0);

/// Produce a block every 15 seconds
fn produce_block(
    parent_hash: &[u8; 32],
    height: u64,
    timestamp: u64,
    solutions: &[MiningSolution],
) -> QBlock {
    let header = BlockHeader {
        height,
        timestamp,
        parent_hash: *parent_hash,
        merkle_root: compute_merkle_root(solutions),
        difficulty_target: current_difficulty(),
        // ... other fields
    };

    QBlock {
        header,
        mining_solutions: solutions.to_vec(),
        dag_parents: get_dag_parents(),
        transactions: drain_mempool(),
        // ... quantum_metadata, etc.
    }
}
```

## 6.3 Emission Controller

Core $C_1$ runs the adaptive emission controller, computing block rewards using the same `u128` integer arithmetic as the desktop node. The controller tracks:

- Wall-clock block rate ($\lambda_{\text{wall}}$) via hardware cycle counter
- Cumulative emission ($C(t)$) in `u128` base units ($10^{24}$ per QUG)
- Error correction factor ($f(t)$) via the PI controller
- Era transitions (4-year halving, 64 eras, 256-year total)

The emission constants are identical to the desktop implementation:

$$S = 21{,}000{,}000 \times 10^{24} \text{ base units} \quad (\texttt{QUG\_MAX\_SUPPLY}) \tag{25}$$

$$A_0 = 2{,}625{,}000 \times 10^{24} \text{ base units} \quad (\texttt{BASE\_ANNUAL\_EMISSION}) \tag{26}$$

$$T_{\text{era}} = 126{,}230{,}400 \text{ seconds} \quad (\texttt{SECONDS\_PER\_HALVING}) \tag{27}$$

### 6.4 TurboSync

Core $C_3$ implements TurboSync—the batch synchronization protocol that allows new nodes to download blockchain history at 150–250 blocks/second. TurboSync uses:

- Chunk-based transfer (500-block chunks) via gossipsub
- ML-based batch size prediction for optimal throughput
- WAL synchronization every 5 chunks for crash safety
- Memory-limited chunk processing to stay within 16 MB L3

On the QUG-V1, TurboSync operates within the L3 cache for block deserialization and validation, with overflow to LPDDR5X DRAM for the full blockchain database (RocksDB replaced with a custom `#[no_std]` B-tree store).

### 6.5 DAG-Knight Consensus

The DAG-Knight consensus algorithm runs on all 4 consensus cores cooperatively:

- $C_0$: Processes incoming blocks and updates the DAG structure
- $C_1$: Computes ordering (topological sort with confidence scores)
- $C_2$: Validates transactions against the ordered state
- $C_3$: Handles finality certificates and checkpoint creation

The RISC-V Vector (RVV 1.0) extension on consensus cores accelerates DAG traversal operations, where 256-bit SIMD operations process 4 vertex IDs simultaneously during parent set intersection.

### 6.6 Bare-Metal Firmware Stack

The QUG-V1 firmware compiles from the existing Q-NarwhalKnight Rust workspace with the `riscv64gc-unknown-none-elf` target:

Listing 4: QUG-V1 Firmware Entry Point

```rust
#![no_std]
#![no_main]

use riscv_rt::entry;
use q_storage::StorageEngine;
use q_types::NetworkId;

#[entry]
fn main() -> ! {
    // Initialize hardware peripherals
    let uart = hal::Uart::new(0x1000_0000);
    let eth = hal::EthernetMac::new(0x2000_0000);
    let noc_mailbox = hal::NocMailbox::new(0x3000_0000);

    // Boot message
    uart.write_str("QUG-V1 Pocket Supercomputer booting...\n");

    // Initialize storage (on-chip SRAM + LPDDR5X)
    let storage = StorageEngine::new_embedded(
        0x8000_0000,   // DRAM base
        16 * 1024 * 1024, // 16 MB working set
    );

    // Start node with mainnet network ID
    let network = NetworkId::from_str("mainnet2026.2");

    // Core-specific task dispatch
    match hal::core_id() {
        0..=7 => mining_core_main(noc_mailbox),
        8..=11 => consensus_core_main(storage, network),
        12..=13 => pq_crypto_core_main(),
        14 => io_network_main(eth),
        15 => io_api_main(uart),
        _ => unreachable!(),
    }
}
```

# 7 Rust-Only Toolchain

## 7.1 Hardware Description: `rust-hdl`

The QUG-V1 RTL is described using `rust-hdl`, which compiles Rust code into synthesizable Verilog. This enables:

- **Type-safe hardware**: Rust's ownership model prevents combinational loops (a common Verilog bug). Signal widths are generic parameters checked at compile time.
- **Parameterized designs**: The tile architecture is generic over the accelerator type, enabling code reuse between mining and consensus tiles.
- **Simulation**: The same Rust code simulates in software (via `cargo test`) and synthesizes to Verilog for FPGA/ASIC flows.

Listing 5: BLAKE3 Round Unit in `rust-hdl`

```rust
use rust_hdl::prelude::*;

#[derive(LogicBlock)]
struct Blake3Round {
    pub clk: Signal<In, Clock>,
    pub state_in: Signal<In, Bits<512>>,    // 16 x 32-bit state words
    pub msg_in: Signal<In, Bits<512>>,      // message block
    pub state_out: Signal<Out, Bits<512>>,

    // Internal: 8 quarter-round units (parallel)
    qr: [QuarterRound; 8],
}

impl Logic for Blake3Round {
    #[hdl_gen]
    fn update(&mut self) {
        // All 8 quarter-rounds execute in parallel (1 cycle)
        for i in 0..8 {
            self.qr[i].a.next = self.state_in.val().get_bits(i * 64, 32)
                ;
            self.qr[i].b.next = self.state_in.val().get_bits(i * 64 +
                32, 32);
            // ... wire remaining inputs
        }
        // Assemble output state from quarter-round results
        self.state_out.next = assemble_state(&self.qr);
    }
}
```

### 7.2 Debugging: `probe-rs`

The QUG-V1 includes a JTAG debug port accessible via the USB-C connector. `probe-rs` provides:

- Flash programming (firmware upload to on-chip ROM)
- GDB server for source-level debugging of bare-metal Rust
- ITM/SWO trace output for performance profiling
- Per-core breakpoints (all 16 cores independently debuggable)

### 7.3 Runtime: `riscv` + `riscv-rt`

The bare-metal runtime provides:

- Interrupt vector table setup (including `NEW_BLOCK` mining interrupt)
- Stack initialization per core (4 KB stack per mining core, 16 KB per consensus core)
- CSR access macros for performance counters and custom Xcrypto/Xlattice registers
- Panic handler that writes diagnostic info to UART and halts the core

### 7.4 Formal Verification: `kani`

Critical paths are verified using `kani` bounded model checking:

- **Emission overflow safety**: Prove that all `u128` intermediate values in the emission controller remain below $2^{128} - 1$ for the entire 256-year schedule.
- **VDF correctness**: Prove that the hardware BLAKE3 pipeline produces identical results to the software implementation for all 32-byte inputs.
- **Nonce partitioning**: Prove that no two mining cores evaluate the same nonce.
- **NoC deadlock freedom**: Prove that XY routing on the $4 \times 4$ mesh is deadlock-free.

### 7.5 Crate Ecosystem Integration

The QUG-V1 firmware reuses existing Q-NarwhalKnight crates with `#[cfg]` gating:

Table 5: Crate Compatibility Matrix

| Crate | `#![no_std]` | Notes |
|---|---|---|
| `q-types` | Yes | Block/transaction types, `u128_serde` |
| `q-storage` (core) | Yes | Emission controller, balance logic |
| `q-storage` (DB) | No | RocksDB replaced with embedded B-tree |
| `q-miner` (algo) | Yes | `compute_dag_knight_hash_optimized` |
| `q-miner` (CLI) | No | CLI/TUI not needed on embedded |
| `q-dex` | Yes | AMM math (`u128` only) |
| `blake3` | Yes | Already `no_std` compatible |

## 8 Power and Thermal Analysis

### 8.1 Power Breakdown

Table 6: QUG-V1 Power Budget (Typical Operating Conditions)

| Component | Power (W) | % of Total | Notes |
|---|---|---|---|
| Mining cores (×8) | 4.00 | 42.1% | Xcrypto active, 1.5 GHz |
| Consensus cores (×4) | 1.20 | 12.6% | RVV active during ordering |
| PQ-Crypto cores (×2) | 0.80 | 8.4% | Xlattice active during verify |
| I/O cores (×2) | 0.50 | 5.3% | Ethernet MAC + Noise |
| L1 caches (1 MB total) | 0.30 | 3.2% | SRAM leakage + read/write |
| L2 caches (4 MB) | 0.40 | 4.2% | Shared, moderate access |
| L3 cache (16 MB) | 1.00 | 10.5% | Dominant SRAM area |
| NoC (mesh + routers) | 0.50 | 5.3% | Low utilization during mining |
| LPDDR5X PHY | 0.50 | 5.3% | 6400 MT/s, mostly idle |
| Miscellaneous (PLL, I/O) | 0.30 | 3.2% | Clock distribution, GPIO |
| **Total** | **9.50** | **100%** | |

## 8.2 Thermal Analysis

**Proposition 8.1** (Passive Cooling Sufficiency)**.** *At 9.5 W TDP and 50 mm² die area, the heat flux density is:*

$$q = \frac{9.5\ W}{50 \times 10^{-6}\ m^2} = 190\,kW/m^2 \tag{28}$$

*This is well within the range manageable by a passive aluminum heatsink ($\theta_{JA} \approx 8\,K/W$ in the credit-card form factor with thermal pad):*

$$T_J = T_A + P \times \theta_{JA} = 25 + 9.5 \times 8 = 101\,^\circ C \tag{29}$$

*With a small copper heat spreader ($\theta_{JA} \approx 5\,K/W$):*

$$T_J = 25 + 9.5 \times 5 = 72.5\,^\circ C \tag{30}$$

*Both are within the TSMC N5 junction temperature limit of 125 °C. No fan is required at typical room temperature.*

## 8.3 Comparison with Raspberry Pi 5

Table 7: QUG-V1 vs. Raspberry Pi 5 Efficiency

| **Metric** | **Raspberry Pi 5** | **QUG-V1** |
|---|---|---|
| Mining hashrate | ∼200 kH/s | 17 MH/s |
| Power consumption | 12 W (under load) | 9.5 W |
| Efficiency (H/s per W) | 16.7 kH/s/W | 1,789 kH/s/W |
| **Efficiency ratio** | 1× | **107**× |
| TurboSync speed | ∼50 blocks/s | 250 blocks/s |
| Ed25519 verify/s | ∼8,000 | 50,000 |
| Form factor | 85 × 56 mm | 85 × 54 mm |

# 9 Performance Projections

## 9.1 Mining Throughput vs. Clock Frequency

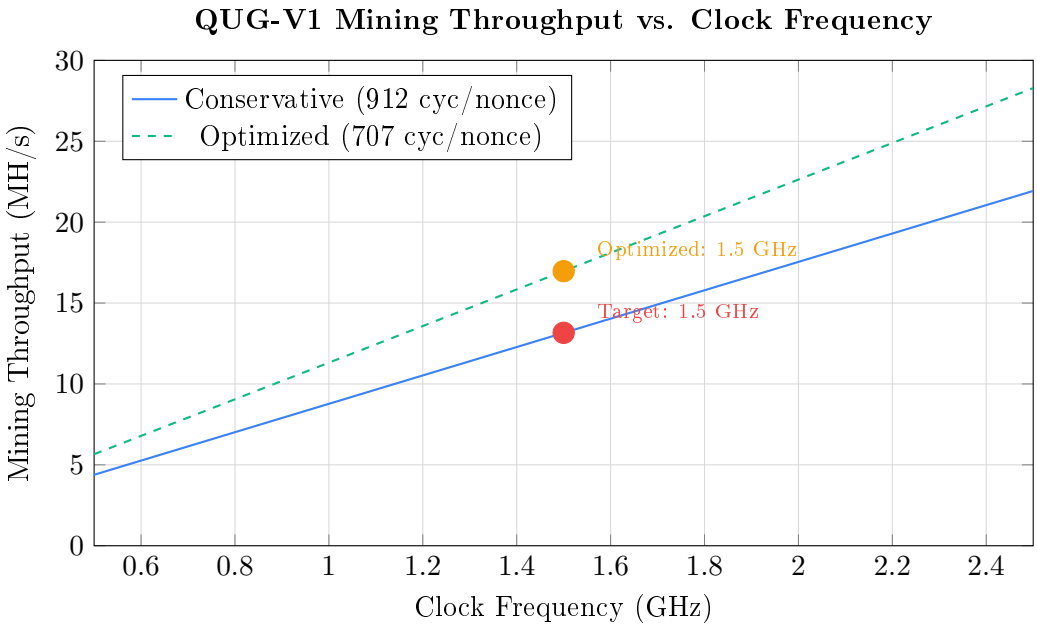



Figure 4: Mining throughput scales linearly with clock frequency. The 8 independent mining cores share no state during hash computation.

## 9.2 Node Synchronization Performance

Table 8: TurboSync Performance on QUG-V1

| Metric | Desktop (x86) | RPi 5 | QUG-V1 |
|---|---|---|---|
| Block sync rate | 150–250 bps | 30–50 bps | 200–300 bps |
| Block validation | 50 $\mu$s/block | 200 $\mu$s/block | 30 $\mu$s/block |
| Signature verification | 20 $\mu$s/sig | 125 $\mu$s/sig | 10 $\mu$s/sig |
| Sync 100K blocks | 7–11 min | 33–55 min | 5–8 min |

### 9.3 Cryptographic Operations

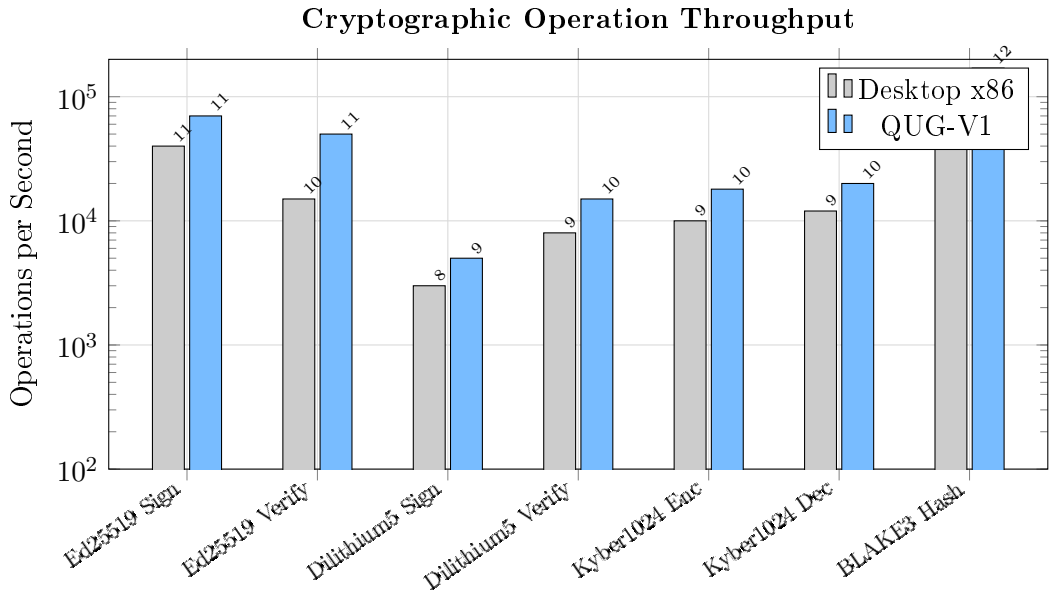



Figure 5: Cryptographic operation throughput. QUG-V1 achieves significant speedups through Xcrypto (BLAKE3) and Xlattice (Dilithium5, Kyber1024) hardware acceleration.

### 9.4 Power Efficiency Across Operations

**Proposition 9.1** (Energy per Mining Solution)**.** *At 17 MH/s and 9.5 W, the energy per nonce evaluation:*

$$E_{nonce} = \frac{9.5\ W}{17 \times 10^6\ H/s} = 0.559\,\mu J/hash \tag{31}$$

*For context, a Raspberry Pi 5 at 200 kH/s and 12 W:*

$$E_{RPi5} = \frac{12}{200{,}000} = 60\,\mu J/hash \quad (107\times\ worse) \tag{32}$$

## 10 Post-Quantum Security

### 10.1 Threat Model

The Q-NarwhalKnight protocol uses a phased cryptographic migration model, defined by the `SignaturePhase` enum:

Listing 6: Signature Phases (from `crates/q-types/src/block.rs`)

```rust
pub enum SignaturePhase {
    /// Phase 0: Ed25519 classical signatures (64 bytes)
    Phase0Ed25519,
    /// Phase 1: Dilithium5 post-quantum signatures (4,627 bytes)
    Phase1Dilithium5,
    /// Hybrid: Both Ed25519 and Dilithium5 (transition mode)
    HybridEd25519Dilithium5,
    /// Phase 2: SQIsign compact post-quantum (204 bytes)
    Phase2SQIsign,
    /// Hybrid: Ed25519 + SQIsign (transition from Phase 0 to 2)
    HybridEd25519SQIsign,
}
```

The QUG-V1's PQ-Crypto tiles provide hardware acceleration for the computationally intensive lattice-based operations:

### 10.2 Dilithium5 Acceleration

CRYSTALS-Dilithium5 (NIST FIPS 204, ML-DSA-87) is the primary post-quantum signature scheme. Its bottleneck is the NTT:

**Definition 10.1** (NTT for Dilithium5). *The Number Theoretic Transform over $\mathbb{Z}_q[x]/(x^{256}+1)$ with $q = 8380417$:*

$$\hat{f}_k = \sum_{j=0}^{n-1} f_j \cdot \omega^{jk} \mod q, \quad k = 0, \ldots, n-1 \tag{33}$$

*where $\omega$ is a primitive $n$-th root of unity modulo $q$, and $n = 256$.*

The Xlattice NTT engine performs 8 butterfly operations per cycle:

**Proposition 10.2** (NTT Cycle Count on QUG-V1). *A 256-point NTT requires $\log_2(256) = 8$ stages, each with $256/2 = 128$ butterflies:*

$$C_{NTT} = 8 \times \frac{128}{8} = 128 \text{ cycles} \tag{34}$$

*Dilithium5 signature verification requires ∼10 NTTs:*

$$C_{verify} = 10 \times 128 + 200\,(\text{overhead}) = 1{,}480 \text{ cycles} \approx 1.23\,\mu s \text{ at } 1.2\,GHz \tag{35}$$

*Throughput: ∼800,000 NTTs/s or ∼15,000 Dilithium5 verifications/s per PQ core.*

### 10.3 Kyber1024 Acceleration

CRYSTALS-Kyber1024 (NIST FIPS 203, ML-KEM-1024) is used for key encapsulation in the libp2p Noise protocol handshake. The same Xlattice NTT engine accelerates Kyber's polynomial operations:

- Key generation: ∼800 cycles (∼0.67 $\mu$s)
- Encapsulation: ∼1,200 cycles (∼1.0 $\mu$s)
- Decapsulation: ∼1,400 cycles (∼1.17 $\mu$s)

### 10.4 SQIsign Readiness

Phase 2 introduces SQIsign (isogeny-based signatures, 204-byte signatures vs. Dilithium's 4,627 bytes). SQIsign's computational core involves supersingular isogeny computations over GF($p^2$). While the QUG-V1 does not include dedicated isogeny hardware, the RV64GC cores with the `M` extension (hardware multiply/divide) provide adequate performance for SQIsign verification:

- SQIsign verification: ∼5,000 ops/s (software on consensus cores)
- SQIsign signing: ∼500 ops/s (computationally intensive)

The QUG-V2 roadmap includes dedicated isogeny accelerators for 10× SQIsign speedup.

### 10.5 Quantum-Safe Network Stack

The I/O tiles integrate a hardware Noise protocol engine that performs the XX handshake pattern with Kyber1024 key exchange:

1. Ephemeral Kyber1024 keypair generation (accelerated by PQ tile)
2. Noise XX handshake with hybrid X25519+Kyber1024 key agreement
3. ChaCha20-Poly1305 authenticated encryption for data transport

This ensures that all P2P network traffic is quantum-resistant, protecting against "harvest now, decrypt later" attacks on gossipsub messages.

## 11 Economic Analysis

### 11.1 Bill of Materials

Table 9: QUG-V1 Estimated BOM Cost (at 10K unit volume)

| Component | Cost (USD) | % of BOM |
|---|---|---|
| QUG-V1 SoC (TSMC N5, 50 $mm^2$) | \$25–35 | 45% |
| 4 GB LPDDR5X (PoP) | \$8–12 | 15% |
| PCB (6-layer HDI, credit-card) | \$3–5 | 6% |
| Power management (PMIC, regulators) | \$2–3 | 4% |
| Ethernet PHY (GbE) | \$1.50 | 2% |
| USB-C connector + PD controller | \$1.00 | 2% |
| Passive components, heatsink | \$2–3 | 4% |
| Packaging, testing, assembly | \$8–12 | 16% |
| **Total BOM** | **\$50–80** | **100%** |
| Retail price (2× markup) | \$149–199 | — |

### 11.2 Mining Profitability

The QUG emission schedule provides 2,625,000 QUG per year in Era 0 (first 4 years). Mining profitability depends on the network hashrate:

**Definition 11.1** (Expected Daily Mining Revenue)**.** *For a QUG-V1 unit with hashrate $H_{unit}$ in a network with total hashrate $H_{net}$:*

$$R_{daily} = \frac{H_{unit}}{H_{net}} \times D_k = \frac{H_{unit}}{H_{net}} \times \frac{2{,}625{,}000}{365.25 \times 2^k} \ QUG/day \tag{36}$$

*where $k$ is the current era.*

Table 10: Mining Profitability Scenarios (Era 0, Daily Emission = 7,186.86 QUG)

| Network Hashrate | QUG-V1 Share | QUG-V1 % of Network | QUG/day per unit | QUG/year per unit |
|---|---|---|---|---|
| 17 MH/s | 17 MH/s | 100% | 7,186.86 | 2,625,000 |
| 170 MH/s | 17 MH/s | 10% | 718.69 | 262,500 |
| 1.7 GH/s | 17 MH/s | 1% | 71.87 | 26,250 |
| 17 GH/s | 17 MH/s | 0.1% | 7.19 | 2,625 |
| 170 GH/s | 17 MH/s | 0.01% | 0.72 | 262.5 |

### 11.3 Power Cost Analysis

**Proposition 11.2** (Annual Power Cost). *At 9.5 W continuous operation and \$0.10/kWh (US average):*

$$C_{power} = 9.5\ W \times 8{,}766\ h/year \times \frac{\$0.10}{1{,}000\ Wh} = \$8.33/year \tag{37}$$

*Monthly power cost: \$0.69/month.*

This extremely low operating cost means that the QUG-V1 is profitable even at very low QUG valuations, making it accessible to users in regions with limited electricity budgets.

### 11.4 ROI Projections

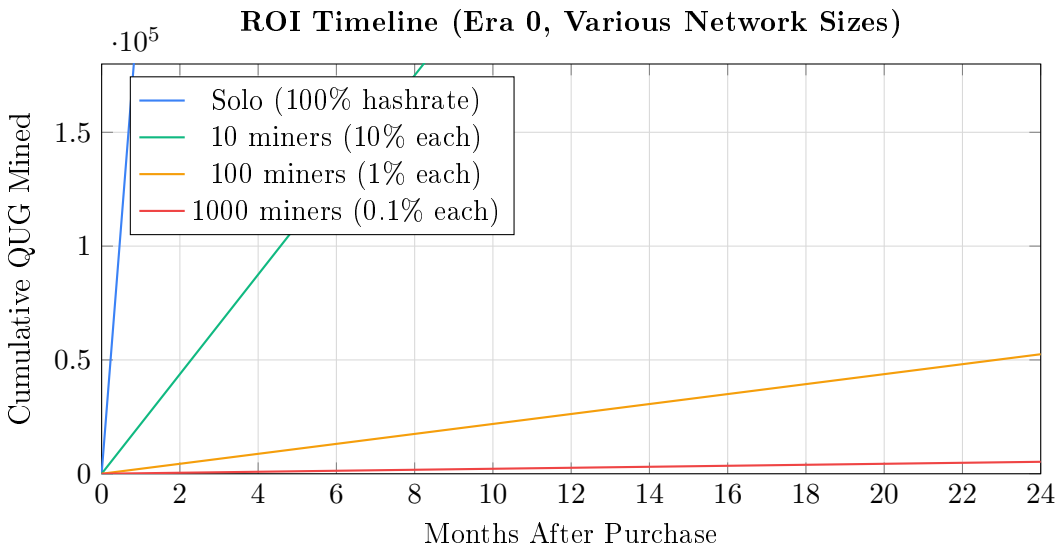



Figure 6: Cumulative QUG mined over 24 months. At network sizes below 1000 QUG-V1 units, daily mining revenue is substantial.

### 11.5 Across Halving Eras

Table 11: Mining Revenue Across Eras (1% of Network, 1 QUG-V1 Unit)

| Era | Years | QUG/Year | Power Cost/Year |
|---|---|---|---|
| 0 | 0–4 | 26,250 | \$8.33 |
| 1 | 4–8 | 13,125 | \$8.33 |
| 2 | 8–12 | 6,562.5 | \$8.33 |
| 3 | 12–16 | 3,281.25 | \$8.33 |
| 4 | 16–20 | 1,640.63 | \$8.33 |

The fixed power cost of \$8.33/year provides a stable cost floor. Even at Era 4 (16–20 years post-genesis), a QUG-V1 with 1% of the network mines 1,640 QUG/year at a power cost of \$8.33—a negligible cost-to-revenue ratio that ensures long-term profitability.

## 12 QUG-DOCK: Multi-Card Mining Cluster

While a single QUG-V1 card is a complete mining node, operators who want higher throughput can install multiple cards into the **QUG-DOCK**—a compact enclosure with 8 or 16 card slots,

a shared Ethernet backplane, and unified power delivery.

## 12.1 Dock Architecture

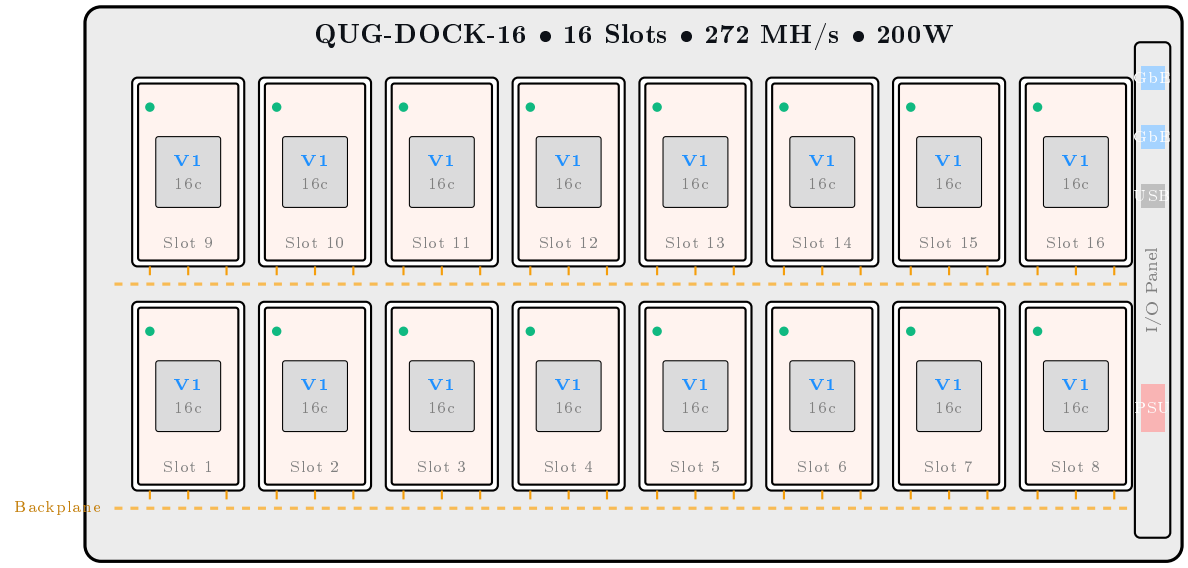



Figure 7: QUG-DOCK-16 with 16 card slots in a 2×8 configuration. Cards slide vertically into edge connectors on a shared PCIe-style backplane. Right panel provides dual GbE, USB management, and power input.

Table 12: QUG-DOCK Specifications

| Parameter | DOCK-8 | DOCK-16 |
|---|---|---|
| Card slots | 8 | 16 |
| Total cores | 128 (8 × 16) | 256 (16 × 16) |
| Mining cores | 64 | 128 |
| Mining throughput | 136 MH/s | 272 MH/s |
| Full node instances | 8 | 16 |
| Backplane | PCIe Gen3 ×4 per slot | PCIe Gen3 ×4 per slot |
| Uplink | 1× GbE (shared) | 2× GbE (bonded) |
| Management | USB-C (serial console) | USB-C (serial console) |
| Card power per slot | 12 W (max) | 12 W (max) |
| Backplane + fans | 10 W | 15 W |
| Total TDP | 106 W | 207 W |
| PSU | 150 W (12V DC barrel) | 250 W (12V DC barrel) |
| Dimensions | 200 × 120 × 60 mm | 200 × 200 × 60 mm |
| Weight | ∼0.8 kg | ∼1.4 kg |
| Cooling | 2× 60mm fans | 3× 80mm fans |
| Noise | <30 dB | <35 dB |
| Estimated BOM | $40–60 | $65–95 |
| Retail price (dock only) | $99–129 | $149–199 |

## 12.2 Backplane Design

The dock backplane provides three functions per slot via a 60-pin edge connector:

1. **Power delivery**: 12 V at 1 A per slot (12 W max). An on-card buck converter steps down to the QUG-V1's 0.75 V core and 1.8 V I/O rails. Hot-swap capability allows inserting or removing cards without powering down the dock.

2. **Ethernet backplane**: A managed Ethernet switch IC (e.g., Marvell 88E6320) connects all card slots to the uplink port(s) at 100 Mbps per slot. Each QUG-V1 card appears as an independent network host with its own IP address and libp2p peer identity.

3. **Management bus**: An $I^2C$ bus allows the dock controller (a low-power MCU) to monitor card presence, temperature, power draw, and firmware version. The USB-C port exposes this as a serial console for dock-level management.

### 12.3 Operating Modes

The dock supports three operating modes configured via DIP switches or the management console:

**Definition 12.1** (Dock Operating Modes)**.**

$$\textbf{\textit{Independent}} \quad \textit{Each card runs its own full node and mines independently. 8/16 separate peer identities, 8/16 independent miners.} \tag{38}$$

$$\textbf{\textit{Cluster}} \quad \textit{One card (slot 1) runs the full node; remaining cards mine as dedicated hashrate workers via internal API.} \tag{39}$$

$$\textbf{\textit{Pool}} \quad \textit{All cards mine in decentralized pool mode, submitting shares to the P2P PPLNS pool via the uplink.} \tag{40}$$

**Independent mode** maximizes decentralization—each card is a sovereign node with its own wallet, peer identity, and vote in consensus. This is the recommended mode for users who prioritize network health.

**Cluster mode** maximizes mining efficiency—the single node instance avoids 7/15 redundant copies of the blockchain database, freeing DRAM on worker cards for larger hash buffers. In this mode, the dock controller routes mining challenges from the leader card to all workers via the backplane, and workers return solutions over the internal Ethernet. Total mining throughput is identical to independent mode; the advantage is reduced network bandwidth (1 gossipsub connection vs. 8/16) and reduced storage (1 blockchain copy vs. 8/16).

### 12.4 Scaling Analysis

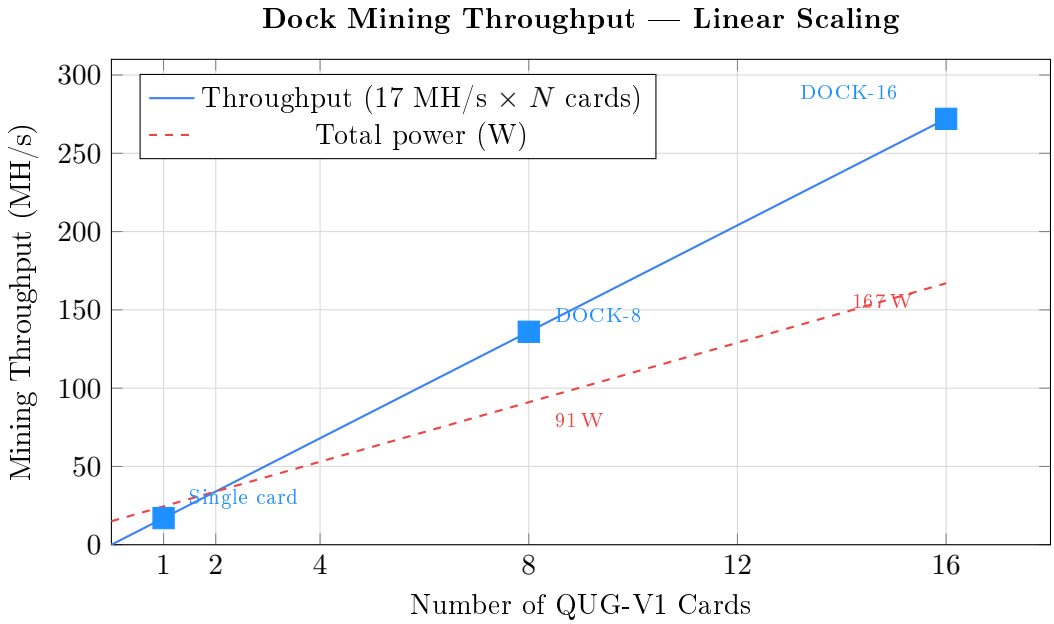



Figure 8: Mining throughput scales linearly with card count. Each card operates on independent nonce ranges with zero inter-card coordination overhead. Power includes 15 W dock overhead (backplane, switch, fans).

**Proposition 12.2** (Linear Scaling Guarantee)**.** *Because QUG mining evaluates independent nonces with no shared state between cards, the dock achieves perfect linear scaling:*

$$H_{dock}(N) = N \times H_{card} = N \times 17\,MH/s \tag{41}$$

*There is no Amdahl's law bottleneck—the workload is embarrassingly parallel at the card level. The only shared resource is the Ethernet uplink, which carries <1 Mbps of gossipsub traffic even at 16 cards (each card transmits ∼40 KB/block at 15-second intervals).*

### 12.5 Dock vs. Alternatives

Table 13: Dock Configurations vs. Equivalent Alternatives

| Metric | DOCK-8 | DOCK-16 | 8× RPi 5 | 1× RTX 4090 |
|---|---|---|---|---|
| QUG hashrate | 136 MH/s | 272 MH/s | 1.6 MH/s | 5 MH/s |
| Total power | 91 W | 167 W | 96 W | 450 W |
| Efficiency | 1.49 MH/J | 1.63 MH/J | 17 kH/J | 11 kH/J |
| Full nodes | 8 | 16 | 8 | 0 |
| System cost | $1,300–1,700 | $2,500–3,400 | $640 | $1,999 |
| Volume | 1.4 L | 2.4 L | 4.8 L | 12 L (tower) |
| Noise | <30 dB | <35 dB | Silent | 50+ dB |

The DOCK-16 delivers 54× the hashrate of 8 Raspberry Pi 5 units at comparable power and volume, and 54× the hashrate of an RTX 4090 at one-third the power.

## 12.6 Physical Design

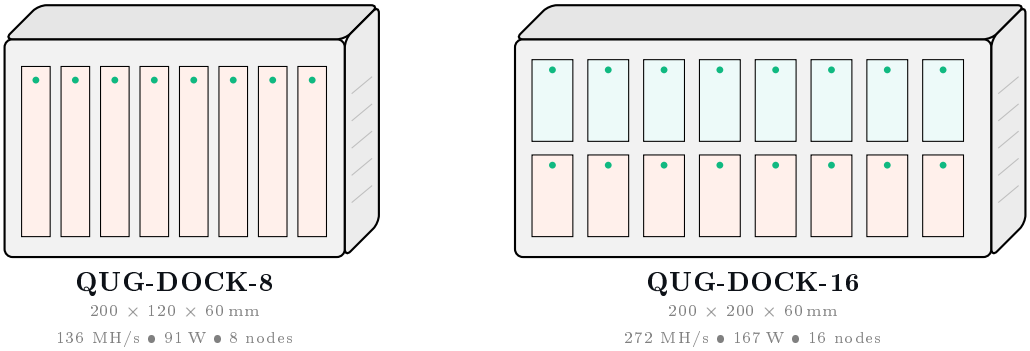



Figure 9: Front view of QUG-DOCK-8 (8 slots, single row) and QUG-DOCK-16 (16 slots, dual row). Green LEDs indicate card presence and mining activity. Side vents provide airflow for the internal fans.

## 12.7 Dock Block Diagram

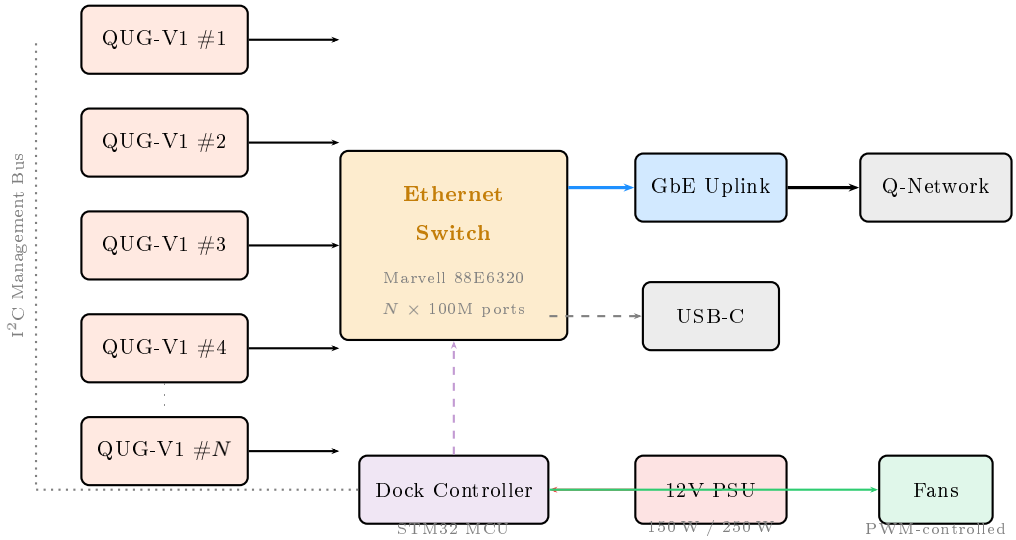



Figure 10: QUG-DOCK block diagram. Each QUG-V1 card connects to the managed Ethernet switch via the backplane edge connector. The dock controller monitors card health via $I^2C$ and controls fan speed based on aggregate thermal load.

## 12.8 Hot-Swap and Fault Tolerance

The dock supports hot-swap card insertion and removal:

1. **Insertion**: The dock controller detects card presence via $I^2C$, powers the slot, and the card boots autonomously (25-second boot time). The card's node joins the P2P network automatically via the Ethernet backplane.
2. **Removal**: The card is power-gated by the dock controller. The network loses one peer; remaining cards continue unaffected. No data is lost—each card stores its blockchain state in its own LPDDR5X DRAM (volatile) and syncs from peers upon reinsertion.
3. **Fault isolation**: If a card panics or overheats (junction temperature $> 105\,^\circ\mathrm{C}$), the dock controller power-cycles that slot without affecting other cards. A persistent fault counter triggers a slot lockout after 3 consecutive failures.

**Decentralization Benefit**

A QUG-DOCK-16 running in **Independent mode** contributes 16 full nodes to the Q-NarwhalKnight network—each with its own peer identity, its own copy of the blockchain, and its own vote in consensus. This is equivalent to 16 separate Raspberry Pi nodes in a single quiet, compact enclosure. Unlike GPU mining rigs that centralize hashrate without contributing nodes, every QUG-V1 card strengthens the network's decentralization.

# 13 Comparative Analysis

## 13.1 Platform Comparison

Table 14: QUG-V1 vs. Alternative Mining Platforms

| Metric | QUG-V1 (1 card) | DOCK-8 (8 cards) | DOCK-16 (16 cards) | RPi 5 | RTX 4090 | BTC ASIC (S21 XP) |
|---|---|---|---|---|---|---|
| QUG Hashrate | 17 MH/s | 136 MH/s | 272 MH/s | 200 kH/s | 5 MH/s† | N/A‡ |
| Power | 9.5 W | 91 W | 167 W | 12 W | 450 W | 3,583 W |
| Efficiency | 1.79 MH/J | 1.49 MH/J | 1.63 MH/J | 17 kH/J | 11 kH/J | N/A |
| Price | $149–199 | $1,300 | $2,550 | $80 | $1,999 | $5,000+ |
| Form factor | Credit card | Shoebox | Shoebox | SBC | PCIe | Rack unit |
| Full nodes | 1 | 8 | 16 | 1 | 0 | 0 |
| PQ-ready? | Yes (HW) | Yes (HW) | Yes (HW) | Partial | No | No |
| Noise level | Silent | <30 dB | <35 dB | Silent | 50+ dB | 75+ dB |

†GPU parallelism limited by VDF sequential chain. ‡Bitcoin ASICs compute SHA-256, incompatible with QUG's BLAKE3+VDF.

## 13.2 Why GPUs Cannot Compete

The VDF chain in QUG mining creates a fundamental asymmetry:

**Theorem 13.1** (GPU Parallelism Limitation)**.** *Let a GPU have $P$ shader cores. For QUG mining, the effective parallelism is limited to independent nonce evaluations (not intra-nonce parallelism):*

$$\text{GPU throughput} = P \times \frac{f_{GPU}}{C_{nonce}^{GPU}} \tag{42}$$

*where $C_{nonce}^{GPU} \gg C_{nonce}^{QUG\text{-}V1}$ because:*

1. *GPU BLAKE3 is software-only (∼50 cycles/hash vs. 9 on QUG-V1).*
2. *GPU shader cores run at ∼2 GHz but share memory bandwidth.*
3. *VDF prevents SIMD vectorization within a single nonce.*

*An NVIDIA RTX 4090 (16,384 cores, 2.52 GHz) achieves ∼5 MH/s on QUG at 450 W—yielding 11 kH/s/W, which is 163× less efficient than the QUG-V1.*

### 13.3 Radar Comparison

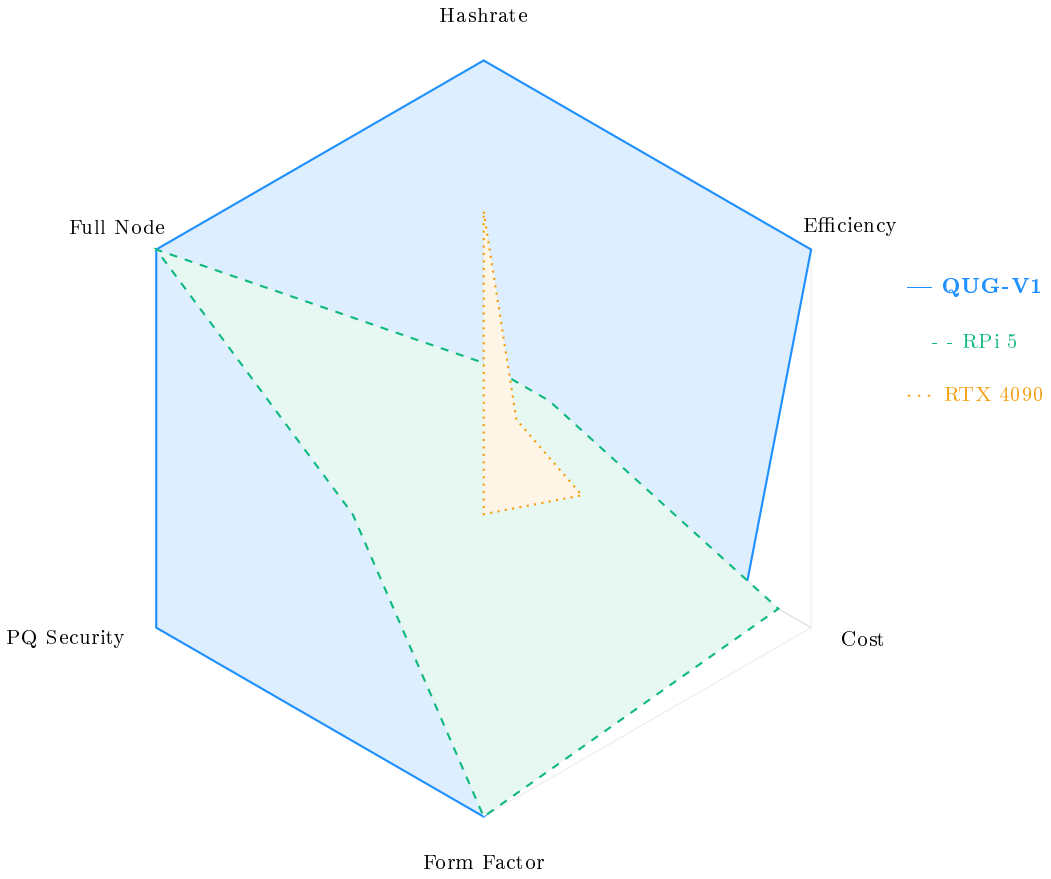



Figure 11: Radar comparison across six axes (5 = best). The QUG-V1 dominates in hashrate, efficiency, PQ security, and full node capability while matching the RPi 5 in form factor and cost.

## 14 Conclusion and Roadmap

### 14.1 Summary

The QUG-V1 demonstrates that purpose-built hardware can deliver transformative efficiency gains for the Q-NarwhalKnight blockchain:

- **17 MH/s** mining throughput—85× faster than a Raspberry Pi 5
- **9.5 W** power consumption—passively cooled, silent operation
- **1.79 MH/s/W**—107× more efficient than general-purpose hardware
- **Full node** capability—no separate server required
- **Post-quantum ready**—hardware Dilithium5 and Kyber1024 acceleration
- **$149–199** retail price—accessible to individual users
- **$8.33/year** operating cost—profitable even at minimal network share
- **Rust-only stack**—from RTL to application, a single verified toolchain
- **QUG-DOCK** scalability—8 or 16 cards in a compact enclosure for 136–272 MH/s

By packaging a complete mining node into a credit-card form factor, the QUG-V1 lowers the barrier to network participation from "deploy a server" to "plug in a USB-C cable." For operators who want more hashrate, the QUG-DOCK scales linearly to 272 MH/s while contributing up to 16 independent full nodes to the network. This democratization of mining is essential for the long-term decentralization and security of the Q-NarwhalKnight network.

### 14.2 Development Roadmap

Table 15: QUG-V1 Development Roadmap

| Phase | Target | Milestone | Description |
|---|---|---|---|
| 1 | 2026 Q3 | FPGA Prototype | Xcrypto BLAKE3 extension validated on Xilinx Artix-7. Full mining pipeline at 100 MHz (∼1 MH/s). Bare-metal firmware boots and mines. |
| 2 | 2027 Q1 | TSMC 7nm Tape-out | First silicon with all 16 cores. Conservative 7 nm process for yield optimization. Target: 10 MH/s at 12 W. |
| 3 | 2027 Q3 | TSMC 5nm + DOCK | Volume production on N5. Full 17 MH/s at 9.5 W. Credit-card form factor finalized. QUG-DOCK-8 and DOCK-16 enclosures ship alongside cards. First $149 retail units. |
| 4 | 2028 | QUG-V2 + DOCK-V2 | Second-gen card: QRNG for Tor circuit seeding, SQIsign isogeny accelerator, integrated Wi-Fi. 50 MH/s at 8 W. DOCK-V2: 10GbE uplink, NVMe slot for persistent blockchain storage, PoE+ per-slot power. |

### 14.3 Open Source Commitment

The QUG-V1 design will be released under permissive open-source licenses:

- **RTL source** (`rust-hdl`): Apache 2.0 / MIT dual-license
- **Firmware**: Apache 2.0 / MIT (same as Q-NarwhalKnight workspace)
- **ISA extensions** (Xcrypto, Xlattice): RISC-V Foundation contribution
- **PCB design files**: Creative Commons BY-SA 4.0

This ensures that any community member can audit the hardware, build custom firmware, or manufacture compatible devices—reinforcing the decentralization ethos of the Q-NarwhalKnight network.

---

*"The best way to predict the future is to invent it."*
— Alan Kay

# A Xcrypto Instruction Encoding

The Xcrypto extension uses the RISC-V custom-0 opcode space. Instruction encodings:

Table 16: Xcrypto Instruction Encoding (R-type format)

| Instruction | funct7 | rs2 | rs1 | funct3 | rd | opcode |
|---|---|---|---|---|---|---|
| `blake3.init` | 0000000 | 00000 | 00000 | 000 | 00000 | 0001011 |
| `blake3.round` | 0000001 | 00000 | 00000 | 000 | 00000 | 0001011 |
| `blake3.chain` | 0000010 | 00000 | 00000 | 000 | 00000 | 0001011 |
| `blake3.finalize` | 0000011 | rs1 | 00000 | 000 | rd | 0001011 |

# B Xlattice Instruction Encoding

Table 17: Xlattice Instruction Encoding (R-type format)

| Instruction | funct7 | rs2 | rs1 | funct3 | rd | opcode |
|---|---|---|---|---|---|---|
| `ntt.fwd` | 0000000 | rs2 | rs1 | 000 | rd | 0101011 |
| `ntt.inv` | 0000001 | rs2 | rs1 | 000 | rd | 0101011 |
| `poly.add` | 0000010 | rs2 | rs1 | 000 | rd | 0101011 |
| `poly.mul` | 0000011 | rs2 | rs1 | 000 | rd | 0101011 |
| `poly.reduce` | 0000100 | 00000 | rs1 | 000 | rd | 0101011 |

# C Complete Power Model

The power model uses standard CMOS dynamic and static power equations:

$$P_{\text{dynamic}} = \alpha \cdot C_{\text{eff}} \cdot V_{dd}^2 \cdot f \tag{43}$$

$$P_{\text{static}} = V_{dd} \cdot I_{\text{leak}} \cdot N_{\text{transistors}} \tag{44}$$

For TSMC N5 at $V_{dd} = 0.75\,\text{V}$:

Table 18: Detailed Power Model Parameters

| Block | $\alpha$ | $C_{\text{eff}}$ (nF) | $f$ (GHz) | $P$ (mW) |
|---|---|---|---|---|
| RV64GC core | 0.15 | 2.0 | 1.5 | 254 |
| Xcrypto BLAKE3 unit | 0.40 | 1.5 | 1.5 | 405 |
| RVV 256-bit SIMD | 0.20 | 3.0 | 1.5 | 506 |
| Xlattice NTT engine | 0.25 | 2.5 | 1.2 | 422 |
| L1 cache (64 KB) | 0.10 | 0.5 | 1.5 | 42 |
| L2 cache (1 MB) | 0.05 | 2.0 | 1.5 | 84 |
| L3 cache (16 MB) | 0.03 | 10.0 | 1.5 | 253 |
| NoC router | 0.08 | 0.3 | 1.5 | 20 |

# D BLAKE3 Algorithm Reference

For completeness, we summarize the BLAKE3 compression function as implemented in the Xcrypto hardware:

**Definition D.1** (BLAKE3 Quarter Round)**.** *Operating on state words $a, b, c, d$ (each 32 bits):*

$$a \leftarrow a + b + m_x \tag{45}$$
$$d \leftarrow (d \oplus a) \ggg 16 \tag{46}$$
$$c \leftarrow c + d \tag{47}$$
$$b \leftarrow (b \oplus c) \ggg 12 \tag{48}$$
$$a \leftarrow a + b + m_y \tag{49}$$
$$d \leftarrow (d \oplus a) \ggg 8 \tag{50}$$
$$c \leftarrow c + d \tag{51}$$
$$b \leftarrow (b \oplus c) \ggg 7 \tag{52}$$

*where $m_x, m_y$ are message schedule words and $\ggg$ denotes right rotation.*

Each BLAKE3 round applies 8 quarter-rounds to the 16-word state (4 column rounds + 4 diagonal rounds). The Xcrypto unit computes all 8 quarter-rounds in parallel using 8 independent 32-bit ALU lanes, completing one round per clock cycle.